

13 장 학습 과제

과제 1. 반복자 범주 비교

`std::forward_list<int>`, `std::list<int>`, `std::deque<int>`, `std::vector<int>`를 만들고 각 컨테이너 반복자에서 사용할 수 있는 이동 연산을 확인한다.

`++`, `--`, `+=`, 반복자 뿔셈, `std::advance()`, `std::distance()`의 사용 가능 여부를 표로 정리한다. 거리 계산 비용이 컨테이너마다 달라지는 이유도 기록한다.

과제 2. 반개방 구간 분할

정수 벡터를 두 부분 범위 `[begin, middle)`과 `[middle, end)`로 나눈다. 앞부분은 오름차순, 뒷부분은 내림차순으로 정렬한다.

원소 수가 홀수일 때 중앙 원소가 어느 범위에 포함되는지 설명하고, 빈 벡터에서도 범위가 올바르게 표현되는지 확인한다.

과제 3. `const_iterator` 구분

수정 가능한 벡터에 대해 `iterator`, `const_iterator`, `iterator const`, `const_iterator const` 변수를 선언한다. 반복자 이동과 원소 수정 가능 여부를 컴파일 주석으로 확인한다.

포인터의 `int*`, `const int*`, `int* const`, `const int* const`와 대응 관계도 작성한다.

과제 4. 삽입 반복자를 이용한 복사

정수 벡터에서 짝수만 골라 빈 `std::vector<int>`에 복사한다. `std::copy_if()`와 `std::back_inserter()`를 사용한다.

같은 원소를 `std::deque<int>`의 앞쪽에 복사하여 입력 순서가 뒤집히는 이유를 설명한다. 순서를 유지하면서 덱 앞쪽에 넣는 방법도 구현한다.

과제 5. 반복 중 안전한 제거

정수 벡터에서 음수를 제거하는 코드를 세 방식으로 작성한다.

- 반복자와 `erase()` 반환값 사용
- `erase-remove` 관용구
- C++20 `std::erase_if()`

각 코드에서 무효화되는 반복자 범위와 순회 중 사용할 반복자를 설명한다.

과제 6. 상태를 가진 함수 객체

기준 점수와 허용 오차를 멤버로 가진 `IsNearScore` 함수 객체를 작성한다. 전달된 점수가 기준 점수에서 허용 오차 이내인지 판정한다.

`std::count_if()`로 조건을 만족하는 점수 수를 계산하고, 같은 동작을 람다 표현식으로도 작성한다.

과제 7. 람다 캡처와 수명

값 캡처, 참조 캡처, 초기화 캡처를 사용하는 람다를 각각 작성한다. 바깥 변수를 변경한 뒤 각 람다의 출력이 어떻게 달라지는지 확인한다.

지역 변수를 참조 캡처한 람다를 함수 밖으로 반환하는 잘못된 코드는 실행하지 않고, 덩글링 참조가 생기는 시점을 주석으로 설명한다.

과제 8. `this`와 `*this` 캡처

멤버 변수 `name`과 `score`를 가진 `Player` 클래스에 두 콜백 생성 함수를 작성한다.

- `[this]`로 현재 객체를 참조하는 콜백
- `[*this]`로 현재 객체를 복사하는 콜백

원본 객체의 값을 변경한 뒤 두 콜백의 결과를 비교한다. 원본 객체가 소멸한 뒤 `[this]` 콜백을 호출할 수 없는 이유도 설명한다.

과제 9. 정렬과 안정 정렬

이름, 점수, 등록 순서를 가진 학생 구조체 벡터를 작성한다. 점수 내림차순으로 `std::sort()`와 `std::stable_sort()`를 각각 적용한다.

동점 학생의 상대 순서를 비교하고, 점수와 이름을 모두 비교하는 함수로 전체 순서를 명시하는 코드도 작성한다.

과제 10. 상위 점수 선택

점수 100개가 저장된 벡터에서 상위 10개를 구한다.

- 전체 `sort()`
- `partial_sort()`
- `nth_element()` 후 앞부분 정렬

세 방식의 결과와 필요한 정렬 범위를 비교한다. 실행 시간을 측정할 때는 난수 입력을 여러 번 생성하고 최적화 설정을 기록한다.

과제 11. 이진 탐색 사전 조건

오름차순 벡터와 내림차순 벡터에서 `lower_bound()`, `upper_bound()`, `binary_search()`를 사용한다.

내림차순 범위에 기본 비교를 사용한 잘못된 호출은 실행 결과에 의존하지 않고 사전 조건 위반으로 설명한다. 정렬과 탐색에 같은 비교 객체를 전달하도록 수정한다.

과제 12. `remove`와 `unique`

중복과 음수가 섞인 정수 벡터에 `remove_if()`와 `unique()`를 적용한다. 각 알고리즘 호출 직후의 `size()`와 논리적 범위를 출력한다.

`erase()`까지 적용한 후 실제 크기를 비교하고, 전체 중복 제거를 위해 정렬이 필요한 이유를 설명한다.

과제 13. 수치 알고리즘

상품 가격과 수량을 별도 벡터에 저장한다. `inner_product()`로 총액을 계산하고, 상품 구조체 벡터에서는 `accumulate()`로 같은 결과를 구한다.

초기값을 `0`, `0.0`, `long long{0}`으로 바꾸어 결과 타입을 비교한다.

과제 14. `accumulate`와 `reduce` 비교

큰 값과 작은 부동소수점 값이 섞인 벡터를 작성한다. `accumulate()`와 `reduce()` 결과를 여러 최적화 설정에서 비교한다.

연산 순서가 결과에 영향을 주는 이유를 부동소수점의 유한 정밀도와 연결해 설명한다.

뿔셈을 축약 연산으로 사용했을 때 결과를 안정적으로 예상할 수 없는 이유도 기록한다.

과제 15. 콜백 이벤트

정수 값을 전달하는 `Event` 클래스를 작성한다. 여러 `std::function<void(int)>` 콜백을 등록하고 식별자로 해제할 수 있게 한다.

통지 중 콜백이 자신을 제거하거나 새 콜백을 추가할 때 발생할 수 있는 반복자 무효화 문제를 분석하고, 변경 요청을 통지 종료 후 처리하는 방식으로 개선한다.

과제 16. `std::function` 선택 보고서

함수 포인터, 함수 템플릿, `std::function`을 사용한 콜백 인터페이스를 각각 작성한다.

캡처 람다 지원, 호출 대상 복사, 실행 파일 코드 증가 가능성, 간접 호출, 동적 할당 가능성, 이동 전용 대상 지원 여부를 비교한다.

과제 17. Ranges Projection

이름, 레벨, 체력을 가진 캐릭터 구조체 벡터를 작성한다. `std::ranges::sort()`와 멤버 포인터 `Projection`을 사용해 레벨 순서와 이름 순서로 각각 정렬한다.

`std::ranges::find()`와 `Projection`으로 특정 이름의 캐릭터를 찾는다. 비교 람다를 직접 작성한 코드와 길이와 역할 분리를 비교한다.

과제 18. View 조합과 지연 평가

정수 벡터에서 양수만 선택하고 제공한 뒤 앞의 다섯 결과만 제공하는 `View`를 작성한다.

`View` 생성 후 원본 벡터의 값을 수정하고 원소를 추가하여 순회 결과가 어떻게 달라지는지 확인한다. 원본 벡터의 재할당이 `View` 순회 중이 아니라 순회 전에 끝나야 하는 이유도 설명한다.

과제 19. View 수명 분석

지역 벡터를 참조하는 `View`를 반환하는 잘못된 함수를 작성하되 반환된 `View`를 실제로 순회하지 않는다. 원본이 소멸하는 시점과 `View`가 무효 상태가 되는 이유를 주석으로 기록한다.

결과 벡터를 반환하는 방식, 원본 범위를 함수 인수로 받는 방식, 소유 객체와 `View`를 함께 보관하는 방식으로 인터페이스를 각각 개선한다.

과제 20. 알고리즘 선택

아래 작업에 적합한 알고리즘이나 직접 반복문을 선택하고 근거를 작성한다.

- 첫 번째 음수 검색
- 모든 체력이 양수인지 판정
- 상위 5개 점수 정렬
- 중앙값 계산
- 정렬된 점수에서 80점 구간 찾기
- 죽은 캐릭터 제거
- 인접한 중복 로그 합치기
- 가격 총합 계산
- 필터, 변환, 개수 제한을 지연 평가로 연결

반복자 범주, 정렬 사전 조건, 시간 복잡도, 추가 메모리, 원본 범위 수명, 반복자 무효화를 각 선택에 반영한다.